STA4026Z Neural Network Assignment - Hitting the Wall.

Jing Yeh
yhxjin001@myuct.ac.za

2026-04-20

Abstract

In this report, we explored the effects of carbohydrate intake, fluid intake, and average speed maintained by runners of both genders on whether they experience the common "hitting the wall" phenomenon during their runs. We used a split-brain neural network that models diet and a runner's profile separately. We found that female athletes can take in a wider range of fluid and carbohydrates than their male counterparts. However, we still recommend a fluid intake of 1 L/h to 1.5 L/h and a carbohydrate intake of 50 g/h to 90 g/h for runners of both genders, assuming they maintain an average speed of 12 km/h.

Keywords: Neural Networks, Sports Science, Athletics Data Analytics

Contents

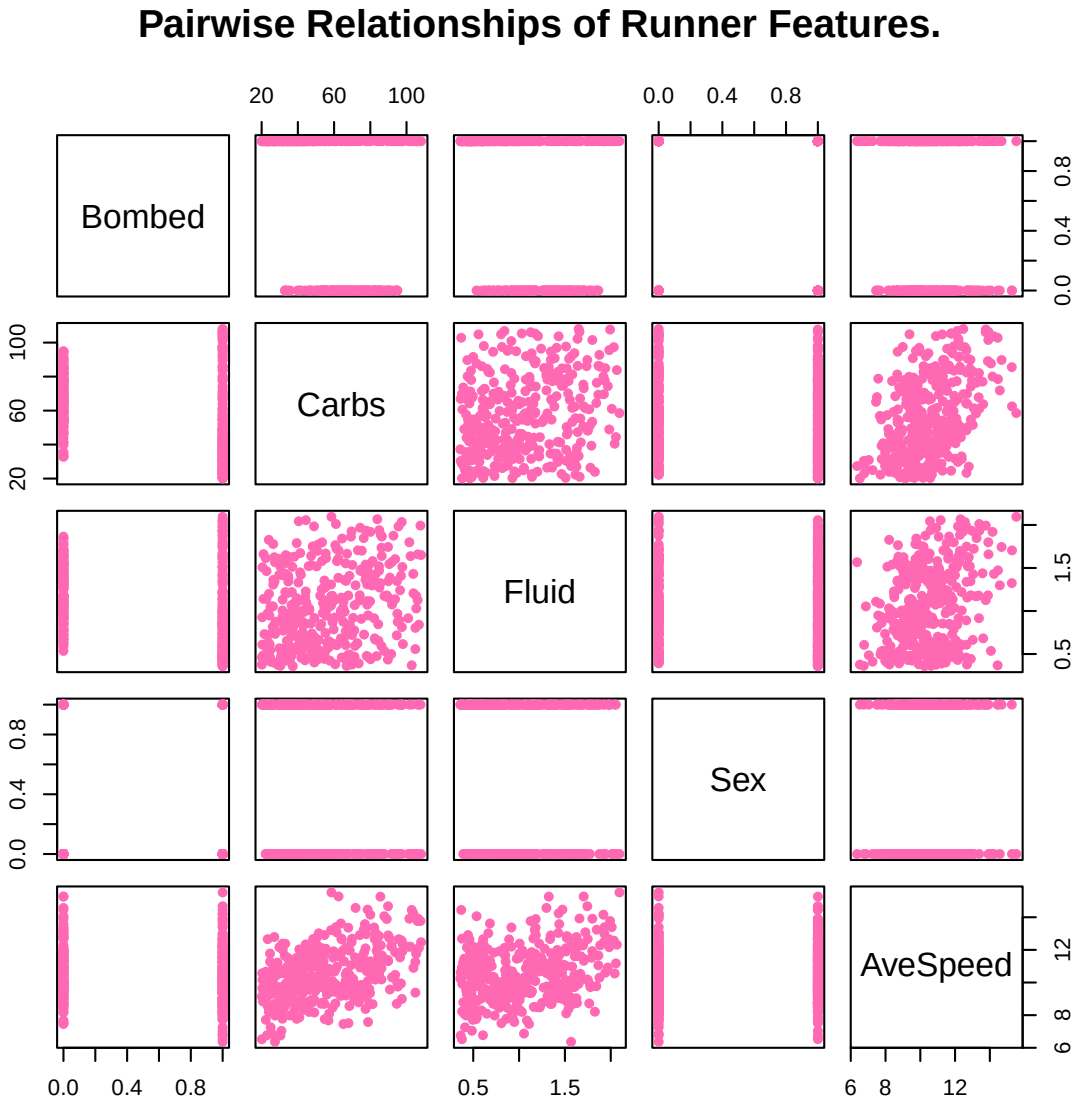
1 Problem Set

1.a . . . . . .
1.b . . . . . .
1.c . . . . . .
1.d . . . . . .
1.e . . . . . .
1.f . . . . . .
1.g . . . . . .
1.h . . . . . .
1.i . . . . . .
1.j . . . . . .
1.k . . . . . .

1 Problem Set

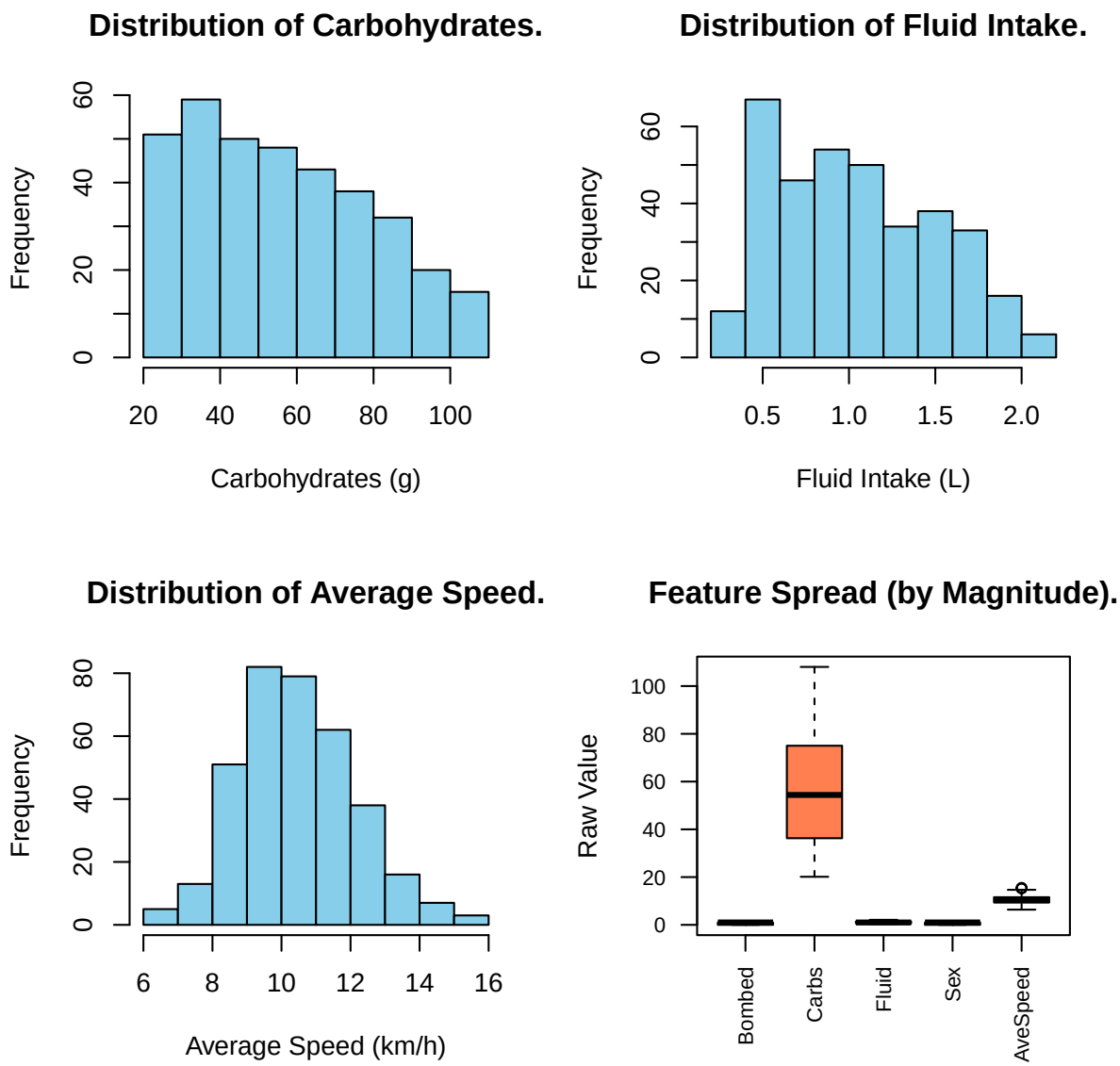
1.a

```
data = as.data.frame(read.table("HittingTheWall_2026.txt", header=TRUE))
pairs(Bombed ~ . , data=data, pch=16, col="hotpink",
      main="Pairwise Relationships of Runner Features.")
```




```
par(mfrow=c(2,2))

# Histograms
hist(data$Carbs, col="skyblue", main="Distribution of Carbohydrates.",
     xlab="Carbohydrates (g)", ylab="Frequency")
```

```r
hist(data$Fluid, col="skyblue", main="Distribution of Fluid Intake.",
     xlab="Fluid Intake (L)", ylab="Frequency")

hist(data$AveSpeed, col="skyblue", main="Distribution of Average Speed.",
     xlab="Average Speed (km/h)", ylab="Frequency")
boxplot(data, col="coral", main="Feature Spread (by Magnitude).",
        ylab="Raw Value", las=2, cex.axis=0.8)
```




```r
par(mfrow = c(1, 1))
```

From the pairwise plots, we can see massive overlap between the classes across every single feature. There is no clear linear threshold in any 2D projection that separates the

runners who bombed from those who did not. This high degree of overlap and the likelihood of complex feature interactions motivates our decision to model the response with a neural network, which is much more flexible in modelling non-linear, multi-dimensional boundaries. Furthermore, this is a classification problem, which motivates our choice of using softmax as our output layer activation funciton.

From the histograms and the boxplots, we see that the features have ranges that vary greatly. This is bad for neural networks. We would have needed to rescale the data if we were using standard gradient descent. However, R's nlm uses a Newton-like second-order derivative optimisation algorithm, which implicitly takes care of the scaling.

1.b

We start with Equation 2:

$$\sigma(z_j) = \frac{\exp(z_j - z^\star)}{\sum_{i=1}^{q} \exp(z_i - z^\star)}.$$

Using the exponent rule $\exp(a - b) = \exp(a)\exp(-b)$, we can expand both the numerator and the denominator:

$$\sigma(z_j) = \frac{\exp(z_j)\exp(-z^\star)}{\sum_{i=1}^{q} [\exp(z_i)\exp(-z^\star)]}.$$

Because the term $\exp(-z^\star)$ is a constant with respect to the summation index i, we can factor it completely out of the sum in the denominator:

$$\sigma(z_j) = \frac{\exp(z_j)\exp(-z^\star)}{\exp(-z^\star)\sum_{i=1}^{q} \exp(z_i)}.$$

We cancel out the $\exp(-z^\star)$ terms in the numerator and denominator, leaving us with Equation 1:

$$\sigma(z_j) = \frac{\exp(z_j)}{\sum_{i=1}^{q} \exp(z_i)}.$$

We do this due to computational constraints. Exponential functions grow quickly as the input value increases, but there is a limit of the number of bits that can be used to represent a number. Hence, we subtract z_j off the max z value to ensure that the kernel is always negative, effectively bounding the values between 0 and 1.

1.c

```
softmax = function(Z_l){
  # This fucntion takes in a q x N matrix,
```

```r
  # with z vector for each observation as columns
  # Outputs a q x N probability matrix

  # We first calculatethe max value for each column
  z_max = apply(Z_l, 2, max)
  # turn the max matrix into a diagonal matrix
  # Use Diagonal to prevent exp(0) = 1
  expZ_max = Diagonal(x = exp(-z_max))
  expZ_l = exp(Z_l)
  expDiff = expZ_l %*% expZ_max

  # Get the denominators
  Denominator = Diagonal(x = 1/colSums(expDiff))

  S = expDiff %*% Denominator
  return(as.matrix(S))
}
```

1.d

Since we are interpreting each observation as row vectors, we can safely define the matrix $X = [X^u X^v]_{N\times(p^u+p^v)}$. In that case, recall that $A_1^u = W_1^{uT} X^{uT} + \vec{b_1^u}$ and $A_1^v = W_1^{vT} X^v + \vec{b_1^v}$, we apply the same logic and define $A_1 = [A_1^u A_1^v]^T_{(d_u^1+d_v^1)\times N}$. Similarly, we define $W_1 = \begin{bmatrix} W_1^u & O \\ O & W_1^v \end{bmatrix}_{(p^u+p^v)\times(d_u^1+d_v^1)}$ and $\vec{b_1} = [b_1^u b_1^v]^T$. We thus have that

$$A_1 = W_1^T X + \vec{b_1}$$

.

We repeat the same procedure to get

$$A_2 = \sigma(W_2^T A_1 + \vec{b_2}),$$

$$A_3 = \sigma_{out}(W_3^T A_2 + \vec{b_3}).$$

1.e

```r
Xu = cbind(data$AveSpeed, data$Sex)
Xv = cbind(data$Carbs, data$Fluid)

# One-hot encoding the response for two outputs as requested
Y = cbind(1 - data$Bombed, data$Bombed)

# I coded the neural network thinking I only need one response,
# so I needed to do this:
#Y = cbind(data£Bombed)
```

1.f

Assume that the output layer has n nodes. The number of **weights** is calculated per layer (W_1, W_2, W_3):

$$\text{Weights} = (p^u m + p^v m) + (2m \times 2) + (2 \times n),$$

$$\text{Weights} = m(p^u + p^v) + 4m + 2n.$$

The number of **biases** is calculated per layer (b_1, b_2, b_3):

$$\text{Biases} = (m + m) + 2 + n,$$

$$\text{Biases} = 2m + 2 + n,$$

For our dataset with m = 4 and two output nodes (n = 2), we have a total of 48 learnable parameters.

1.g

```r
sig = function(A){
  return(tanh(A))
}
#sig_out = function(a){
#  return (1/(1+exp(-a)))
#}
sig_out = function(A){
  return (softmax(A))
}
```

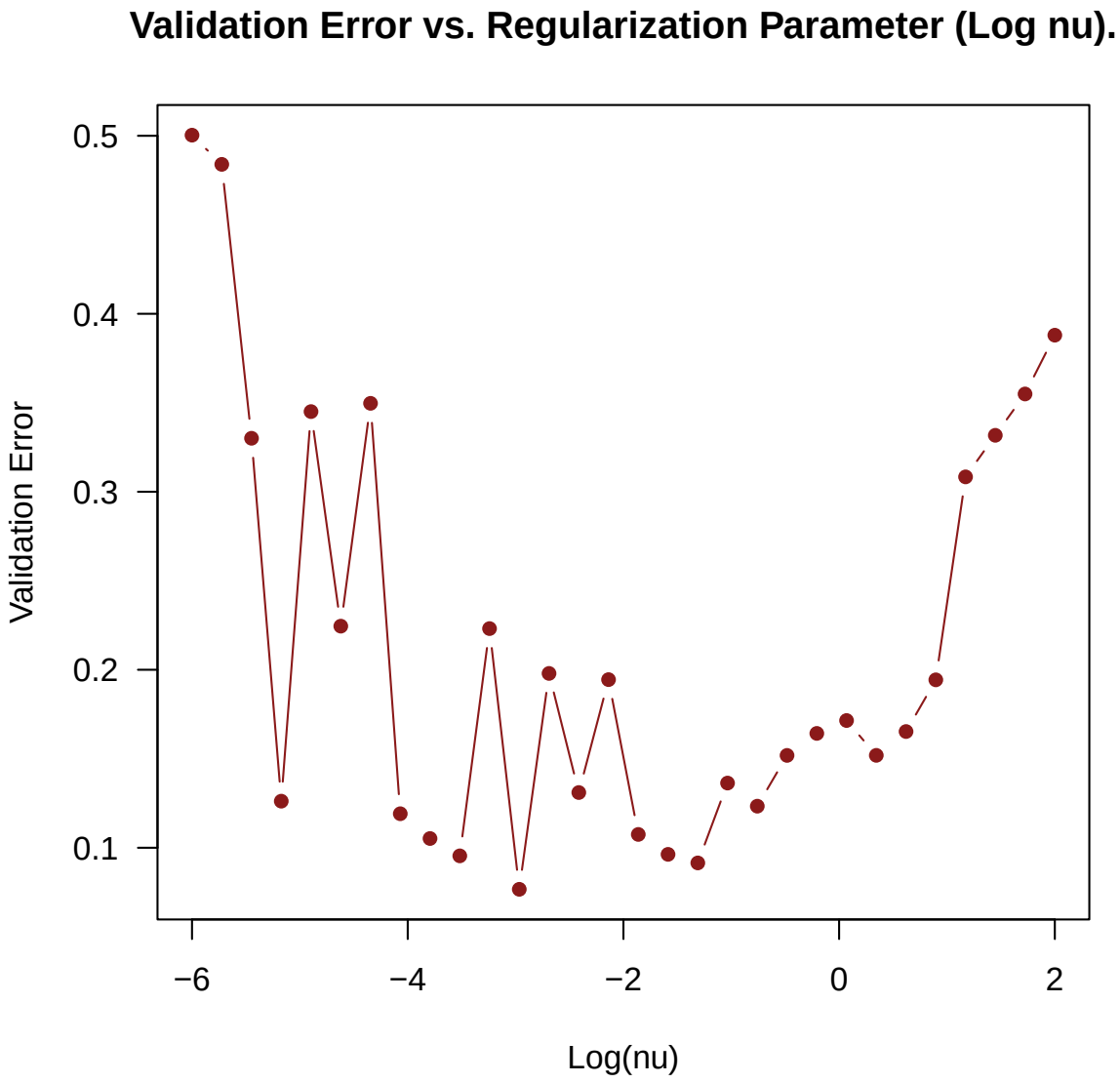
```r
obj = function(Y, Y_hat){
  epsilon = 1e-8 # to prevent log(0)
  Y_hat = pmax(pmin(Y_hat, 1-epsilon),epsilon)
  return (sum(-(Y * log(Y_hat))))
}

neural_net = function(Xu, Xv, m, theta, nu, Y=NULL){
  # output: Y_hat: Raw probability output of the logistic function
  # output: pred: prediction value
  # output: E: evaluation error as required

  pu = dim(Xu)[2]
  pv = dim(Xv)[2]
  N = dim(Xu)[1]
  X = cbind(Xu, Xv)
  # 2m part

  index = 1:(m*pu)
  W1u = matrix(theta[index], pu, m)

  index = max(index) + 1:(pv*m)
  W1v = matrix(theta[index], pv, m)

  W1 = as.matrix(bdiag(W1u, W1v))

  # 2 layer
  index = max(index)+1:(2*m*2)
  W2 = matrix(theta[index], 2 * m, 2)


  index = max(index)+1:4
  W3 = matrix(theta[index], 2, 2)

  index = max(index) + 1:(m)
  b1u = matrix(theta[index], m, 1)
  index = max(index) + 1:(m)
  b1v = matrix(theta[index], m, 1)
  b1 = rbind(b1u, b1v)
```

```r
  index = max(index) + 1:2
  b2 = matrix(theta[index], 2, 1)
  index = max(index) + 1:2
  b3 = matrix(theta[index], 2, 1)

  ones = matrix(1, N, 1)
  A0 = as.matrix(t(X))
  A1 = sig(t(W1) %*% A0 + b1 %*% t(ones))
  A2 = sig(t(W2) %*% A1 + b2 %*% t(ones))
  Z_out = t(W3) %*% A2 + b3 %*% t(ones)
  # Since this is a classification problem, we use logistic function
  Y_hat = t(sig_out(Z_out))
  pred = max.col(Y_hat) - 1
  # Evaluation
  E=NULL
  if (!is.null(Y)){

    base_error = obj(Y, Y_hat) / nrow(Y)
    penalty = (nu / nrow(Y)) * (sum(W1^2) + sum(W2^2) + sum(W3^2))
    E = base_error + penalty
  }

  # only return column 2 (bombed)
  return(list(Y_hat = Y_hat, pred = pred, E=E))
}
```

1.h

```r
set.seed(2026)
m = 4
N = dim(Y)[1]
train_indices = sample(1:N, size=round(0.8*N))

Xtrain_u = Xu[train_indices,]
Xtrain_v = Xv[train_indices,]
Ytrain = as.matrix(Y[train_indices, ])
```

```r
Xvalid_u = Xu[-train_indices,]
Xvalid_v = Xv[-train_indices,]
Yvalid = as.matrix(Y[-train_indices, ])

pu = dim(Xu)[2]
pv = dim(Xv)[2]
nparams = (pu * m) + (pv * m) + 2 * m  + 2 * m * 2 + 2 + 2 + 4
theta = runif(nparams, -1, 1)

nu_grid = exp(seq(-6,2,length
            =30))
# Do grid search on values of nu_grid
errors = rep(1, length(nu_grid))
theta_store = matrix(NA, length(nu_grid), nparams)
for (i in 1:length(nu_grid)){
  nu = nu_grid[i]

  obj_train = function(theta_init) {
    result = neural_net(Xtrain_u, Xtrain_v, 4, theta_init, nu, Ytrain)
    return(result$E)
  }
  cat("Training network for nu =", nu, "\n")
  trained = nlm(obj_train, theta, iterlim=1000)
  theta_store[i, ] =trained$estimate
  valid_result = neural_net(Xvalid_u, Xvalid_v, 4, trained$estimate, nu)

  cat("Finished validating nu =", nu, "\n")
  errors[i] = obj(Yvalid, valid_result$Y_hat) / nrow(Xvalid_u)
}
```

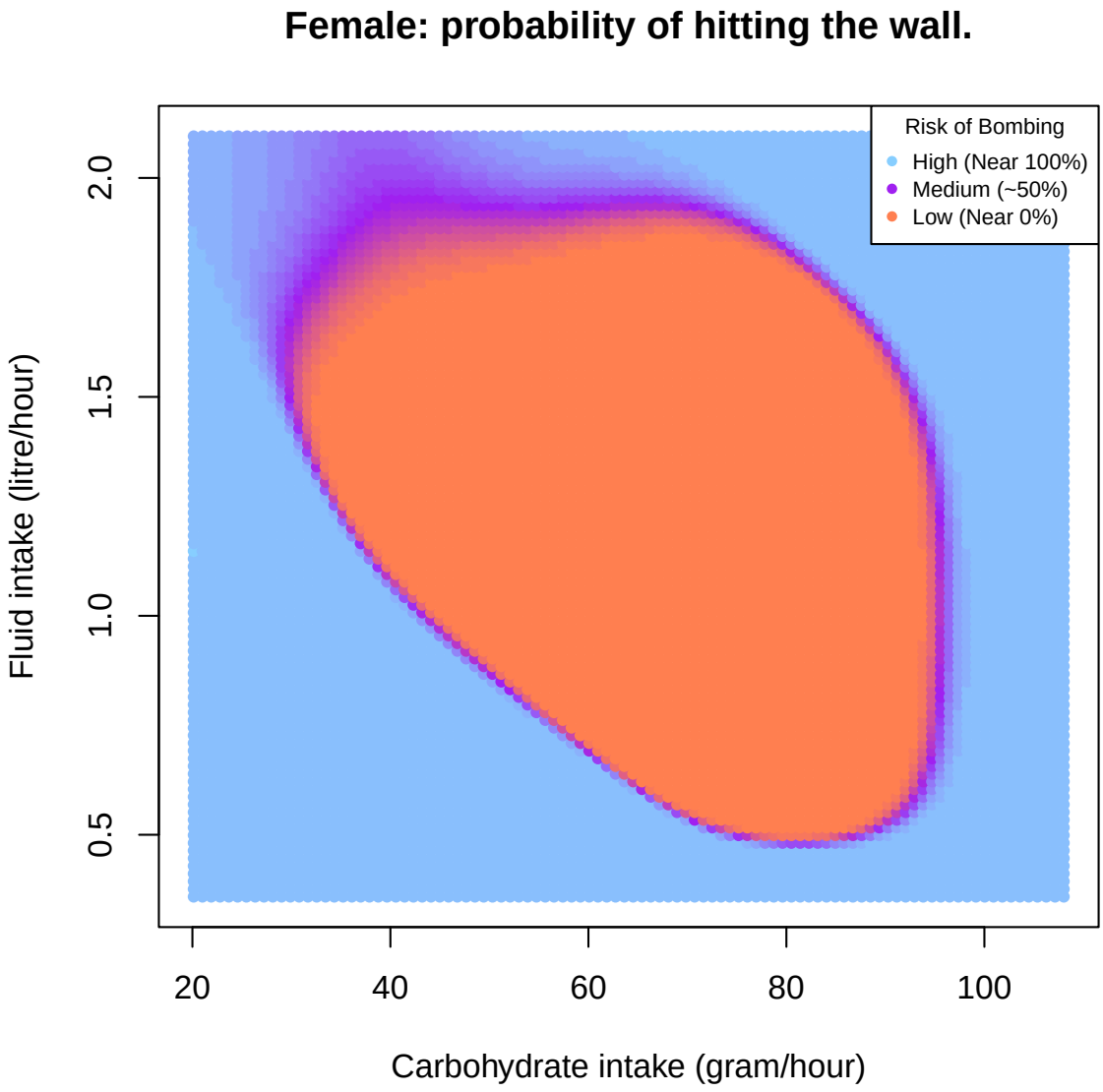


From the above, the best ν value that results in the lowest validation loss is approximately 0.05 (0.0515 using the scale above).

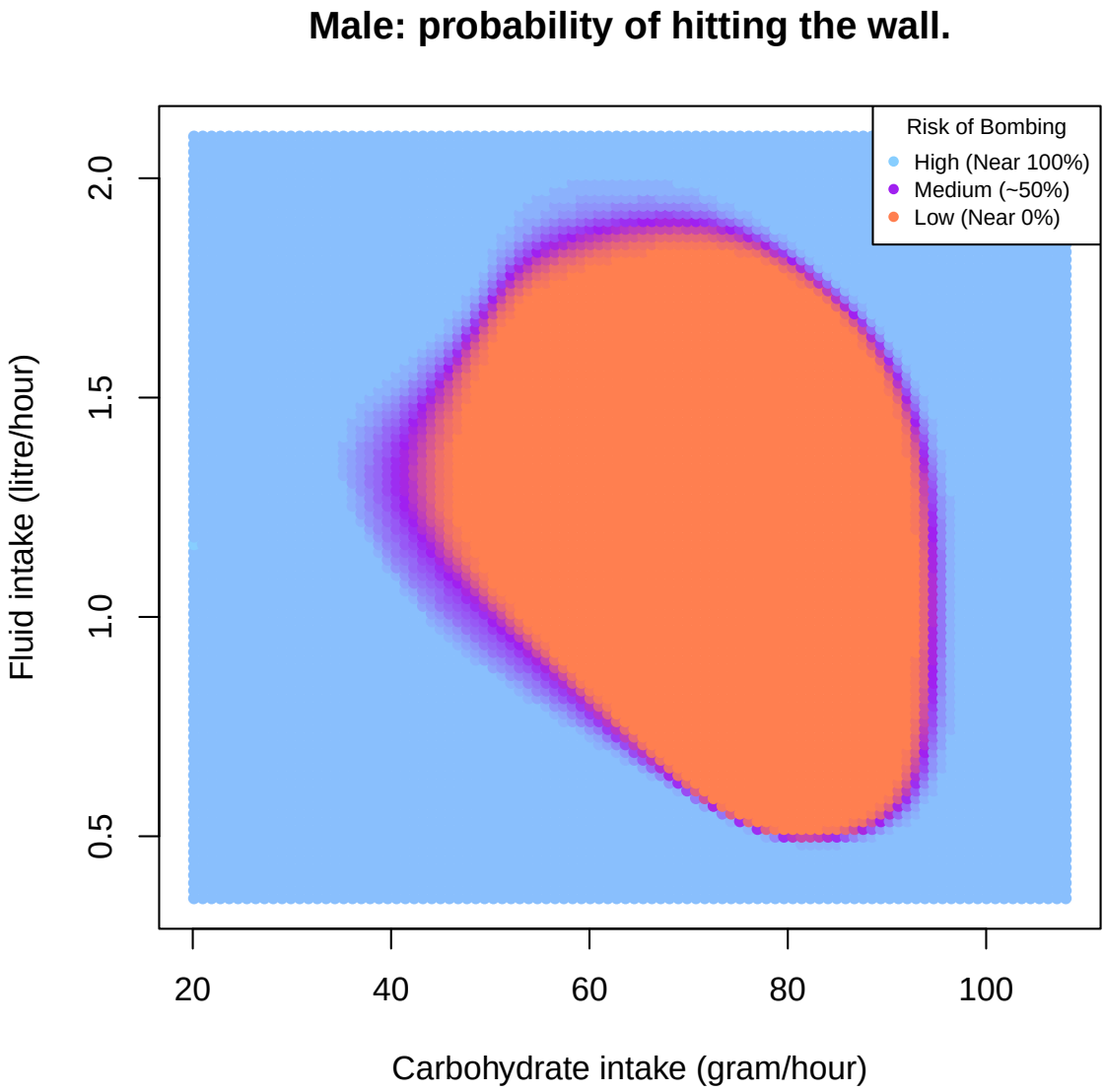
1.i

```
library(colorspace)
color.gradient = function(x, colors=c('coral','purple','skyblue1'), colsteps=25)
{
  colpal = colorRampPalette(colors)
  return( colpal(colsteps)[ findInterval(x, seq(min(x),max(x), length=colsteps)) ] )
}
```

```r
M=100
carbs_seq = seq(min(data$Carbs),max(data$Carbs),length.out= M)
carbs_mean = mean(data$Carbs)
fluid_seq = seq(min(data$Fluid),max(data$Fluid),length.out= M)
fluid_mean = mean(data$Fluid)

xxv1 = rep(carbs_seq, M)
xxv2 = rep(fluid_seq, each =M)
XXv = cbind(xxv1, xxv2)

xxu1 = rep(12, M^2)
xxu2_male = rep(1, each = M^2)
xxu2_female = rep(0, each=M^2)
XXumale = cbind(xxu1, xxu2_male)
XXufemale = cbind(xxu1, xxu2_female)

response_male = neural_net(XXumale, XXv, 4, theta_best_nu, 0)
response_female = neural_net(XXufemale, XXv, 4, theta_best_nu, 0)

plot(xxv2 ~ xxv1, col=color.gradient(response_female$Y_hat[,2]), pch=16, cex=0.8,
     xlab="Carbohydrate intake (gram/hour)", ylab="Fluid intake (litre/hour)",
     main="Female: probability of hitting the wall.")
legend("topright", title="Risk of Bombing",
       legend=c("High (Near 100%)", "Medium (~50%)", "Low (Near 0%)"),
       col=c("skyblue1", "purple", "coral"), pch=16, bg="white", cex=0.7)
```




```
plot(xxv2 ~ xxv1, col=color.gradient(response_male$Y_hat[,2]), pch=16, cex=0.8,
     xlab="Carbohydrate intake (gram/hour)", ylab="Fluid intake (litre/hour)",
     main="Male: probability of hitting the wall.")
legend("topright", title="Risk of Bombing",
       legend=c("High (Near 100%)", "Medium (~50%)", "Low (Near 0%)"),
       col=c("skyblue1", "purple", "coral"), pch=16, bg="white", cex=0.7)
```



1.j

From the response plots, we observe that male runners are more likely to hit the wall running at 12km/h if they are not careful with their diet. The safe region is fairly centred, but we may observe a negative trend: the more carbohydrates consumed, the less fluid is needed to remain safe, and vice versa. There is also the risk of over-consumption and under-consumption in either category. For fluid intake, anything under 0.5 L/h and more than 2 L/h leads to a high risk of bombing. For carbohydrate intake, the lower bound and the upper bound are 40 g/h and 100 g/h respectively.

Female runners, on the other hand, have a significantly larger safety range compared to male athletes. Notably, females can sometimes avoid hitting the wall even at very low carbohydrate intakes (20-40g/h) provided their fluid intake is high (>1.5 L/h). As mentioned, this is a physiological tolerance that males do not share. However, the lower

bound for fluid intake and the upper bound for carbohydrate intake remain the same as male runners.

To summarise, both plots indicate a ceiling effect where over-consuming both fluids and carbohydrates (the top-right region) increases the risk of bombing. As such, while females have more leeway, the safest region for both sexes running at 12km/h remains 1 L/h to 1.5 L/h of fluid and 50g/h to 90g/h of carbohydrates. This is a practical recommendation that we advise runners of both sexes to adhere to, assuming an average running speed of 12km/h.

1.k

The split-brain architecture makes intuitive sense because the features naturally fall into two distinct, independent domains: physiological/demographic states (Xu: Speed, Sex) and active nutritional intake (Xv: Carbs, Fluid). Processing them in separate sub-networks for the first layer allows the model to learn pure representations of a runner's "physical profile" and "nutrition profile" before forcing them to interact.

As calculated in Part (f), the split-brain network explicitly prevents connections between the u and v inputs in the first hidden layer. A standard fully-connected network would calculate weights for every input against every node, drastically increasing the parameter count. By zeroing out the cross-hemisphere weights (using the block-diagonal matrix from Part d), the split-brain network acts as a strict form of architectural regularisation, preventing the model from overfitting to random noise between independent variables.

While appropriate, we must note a limitation based on our findings in Part (j). We discovered some interaction between Sex and nutritional safety zones. Because of the split-brain design, the network is incapable of learning the interaction between "Sex" and "Carbohydrates" in the first hidden layer. It is forced to pass those independent concepts up to the second hidden layer before they can be combined. Fortunately, the network was still successful at modeling this interaction, proving the architecture is deep enough to handle the data despite the initial bottleneck.